

CPSC-406 Report

Sami Hammoud
Chapman University

March 30, 2026

Abstract

Contents

1	Introduction	1
2	Week by Week	1
2.1	Week 1	1
2.2	Week 2: Operations on Automata	4
2.3	Week 3	8
2.4	Week 4	11
2.5	Week 5	12
2.6	Week 6	13
3	Synthesis	16
4	Evidence of Participation	16
5	Conclusion	16

1 Introduction

2 Week by Week

2.1 Week 1

Exercise 1

Consider the following DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$. Complete the following table with all 8 possible strings of length 3 over $\{a, b\}$ (columns: Accepted by $\mathcal{A}_1$? and Accepted by $\mathcal{A}_2$?).

Answer to Exercise 1.1. Here is my completed table (words of length 3 over $\{a, b\}$):

word	Accepted by $\mathcal{A}_1$?	Accepted by $\mathcal{A}_2$?
<i>aaa</i>	No	Yes
<i>aab</i>	Yes	No
<i>aba</i>	No	No
<i>abb</i>	No	No
<i>baa</i>	No	Yes
<i>bab</i>	No	No
<i>bba</i>	No	No
<i>bbb</i>	No	No

Exercise 1.2

Describe the languages $L(\mathcal{A}_k)$ accepted by $\mathcal{A}_k$ for $k = 1, 2$.

Answer to Exercise 1.2. For $k = 1$, $L(\mathcal{A}_1)$ is just the single word *aab*. For $k = 2$, $L(\mathcal{A}_2)$ is all words over $\{a, b\}$ that end in *at least two a's in a row* (so the last two symbols are *aa*).

Exercise 2 (Designing DFAs)

Design DFAs whose accepted languages are given as follows:

1. All the words that end with *ab*
2. All the words that contain *aba*
3. All the words that contain an odd number of *a*'s and an odd number of *b*'s
4. All the words that contain an even number of *a*'s and an odd number of *b*'s
5. All the words such that any three consecutive characters contain at least one *a*
6. All the words that contain *bbb*

What do you notice when comparing the various automata?

Answers for Exercise 2.

(1) **Words that end with *ab*.** I use three states that remember the last one or two symbols. The DFA is:

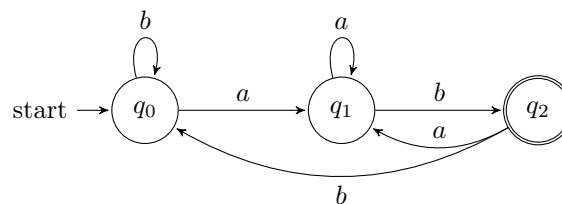


Figure 1: DFA for all words that end with *ab*.

(2) **Words that contain *aba*.** Here I track how much of the pattern *aba* I have just seen as a suffix.

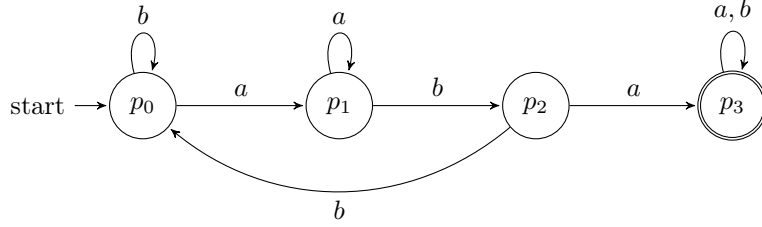


Figure 2: DFA for all words that contain aba as a substring.

(3) Odd number of a 's and odd number of b 's. For this and the next language I use the same 4-state parity automaton, but choose different accepting states. The four states remember the parity (even/odd) of the number of a 's and b 's seen so far.

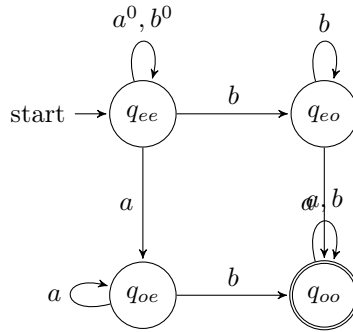


Figure 3: Parity DFA: state q_{xy} means x -parity for a 's and y -parity for b 's. For language (3) we accept q_{oo} (odd a 's and odd b 's).

(4) Even number of a 's and odd number of b 's. Using the same parity DFA in Figure 3, this time the accepting state is q_{eo} (even a 's, odd b 's) instead of q_{oo} .

(5) Any three consecutive characters contain at least one a . This is the same as saying that we never see bbb as a substring. I track runs of consecutive b 's up to length 3; hitting three b 's in a row moves to a dead (rejecting) state.

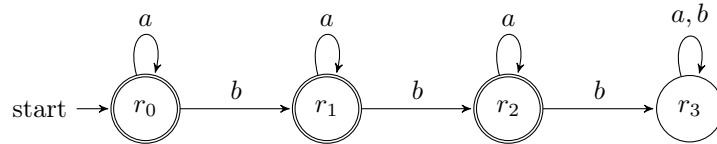


Figure 4: DFA that forbids bbb ; states r_0, r_1, r_2 are accepting, r_3 is a dead state reached after three b 's in a row.

(6) Words that contain bbb . I reuse the same transition structure as in Figure 4, but now r_3 is the *only* accepting state (once we have seen three consecutive b 's we stay in an accepting sink).

What I notice. Several of these automata reuse the same underlying structure but just change which states are accepting (for example (3) vs. (4), and (5) vs. (6)). In other words, a lot of the “different”

languages here really come from the same core DFA with a different choice of final states.

Question

How might you combine the languages $L(\mathcal{A}_1)$ and $L(\mathcal{A}_2)$ from Exercise 1 using set operations (union, intersection, complement)? Could you represent the logic with boolean logic exclusively?

2.2 Week 2: Operations on Automata

Product automata

Exercise 1 (Product automata)

Consider the following automata $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$.

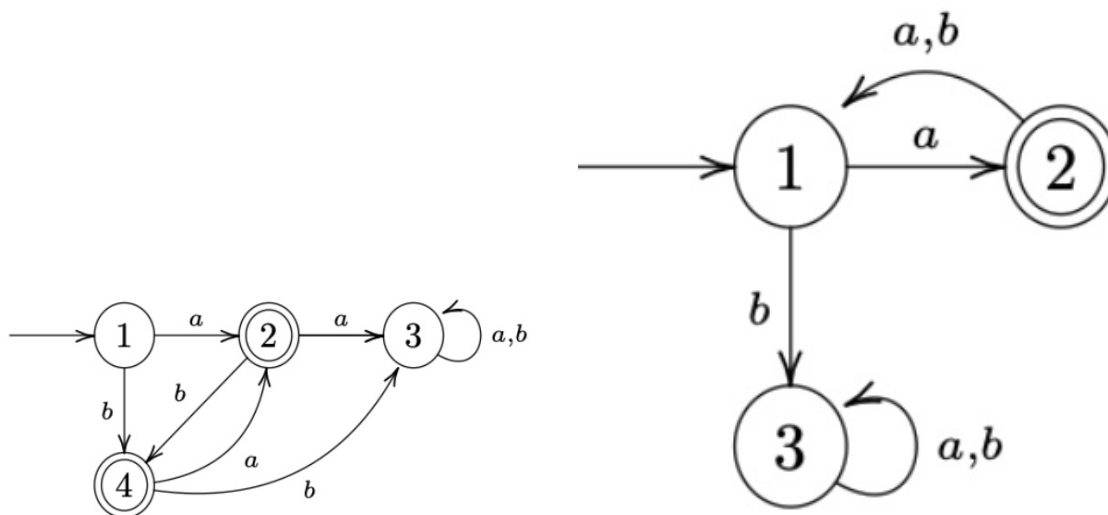


Figure 5: $\mathcal{A}^{(1)}$ (left) and $\mathcal{A}^{(2)}$ (right).

1. Describe $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$.

Answer:

- $L(\mathcal{A}^{(1)})$: Words include $a, b, ba, bab, baba, ab, aba, abab$, etc. I can see that each character must alternate based on its successive character.
- $L(\mathcal{A}^{(2)})$: Words include a, axa (where x doesn't matter). I see the word must start with a , and must add to its word length in increments of 2, every other character being a .

2. Construct the intersection automaton $\mathcal{A}$ for $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$ by drawing its transition diagram (omit unreachable states).

Answer: Below is the intersection automaton diagram.

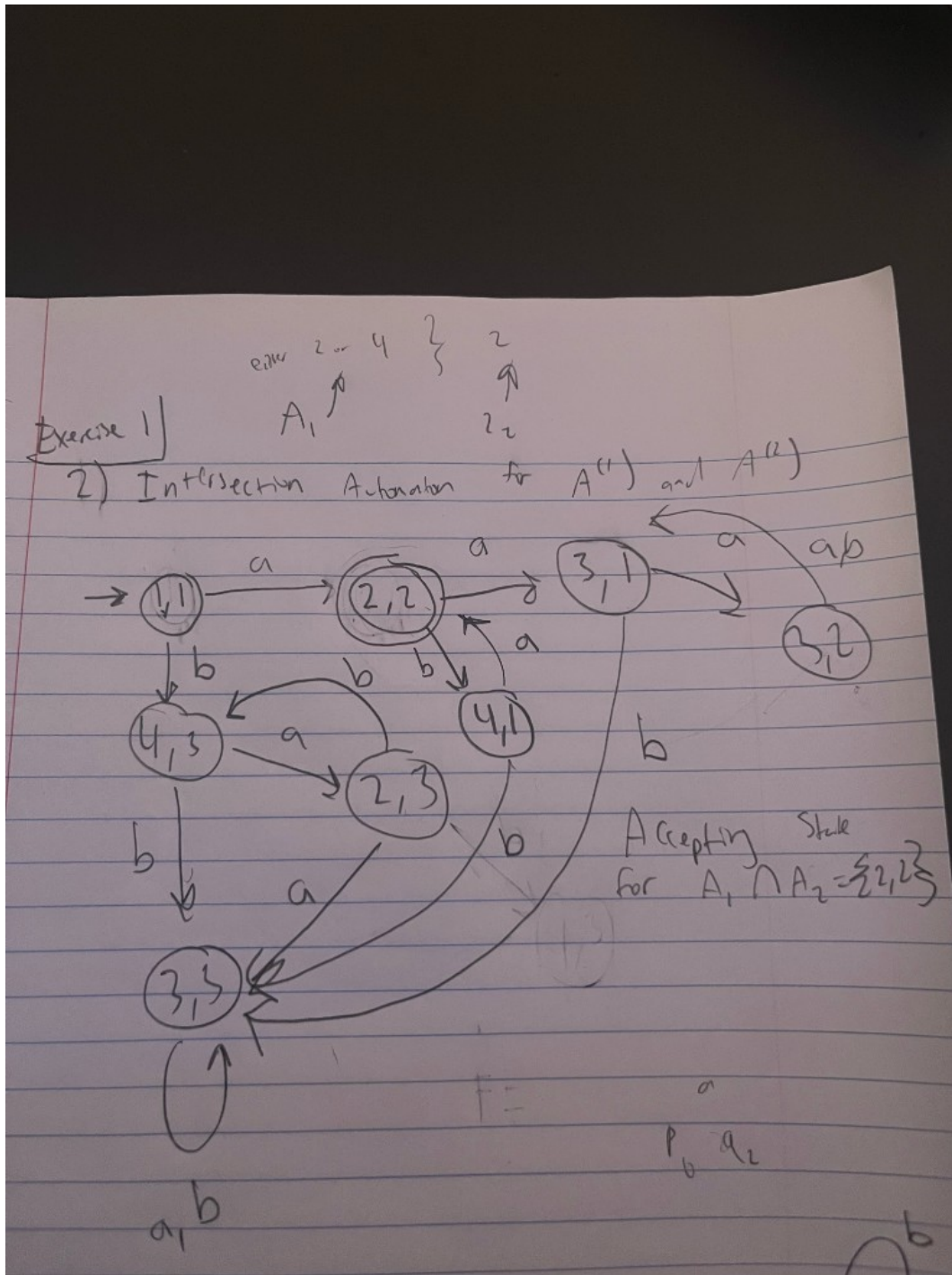


Figure 6: Intersection automaton $\mathcal{A}$ for $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$ (omit unreachable states).

Note: $\{2, 2\}$ as accepting state for intersection product automaton.

3. Argue that $L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

Answer: Since $L(\mathcal{A}^{(1)})$ includes words such as a , b , ba , bab , $baba$ and ab , aba , $abab$, and $L(\mathcal{A}^{(2)})$ includes words that (1) must start with a —so we can disregard the words that start with b in $L(\mathcal{A}^{(1)})$, leaving us with a , ab , aba , etc.—and (2) the requirement for $L(\mathcal{A}^{(2)})$ is to be in increments of two, we are left with patterns: a , aba , $ababa$, etc. Thus we obtain the automaton of both intersection.

4. **How can you change $\mathcal{A}$ to get an automaton $\mathcal{A}'$ with $L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)})$?**

Answer: In the product automaton we would add the accepting states to not be exclusive to both, but inclusive of either one of the subset DFA's accepting state.

Exercise 2 (More automata)

Consider the following automata $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$.

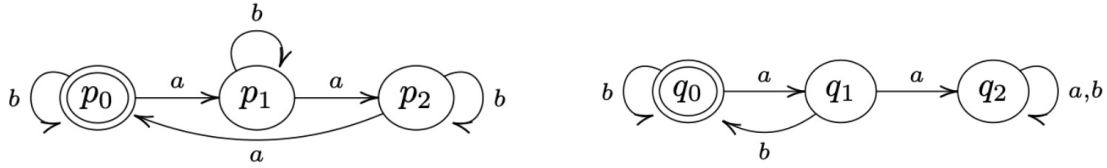


Figure 7: $\mathcal{B}^{(1)}$ (left) and $\mathcal{B}^{(2)}$ (right).

(Recall: the complement of a language $L \subseteq \Sigma^*$ is $\bar{L} := \Sigma^* \setminus L$.)

1. **Describe $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$.**

Answer:

Figure 8: $L(\mathcal{B}^{(1)})$ rules: include 3 a 's at a time, any amount of b 's. $L(\mathcal{B}^{(2)})$ rules: b must follow an a , any amount of b 's. DFA diagram for language 2.

2. **Construct the intersection automaton $\mathcal{B}$ for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ (omit unreachable states).**

Answer: Below is the intersection automaton diagram.

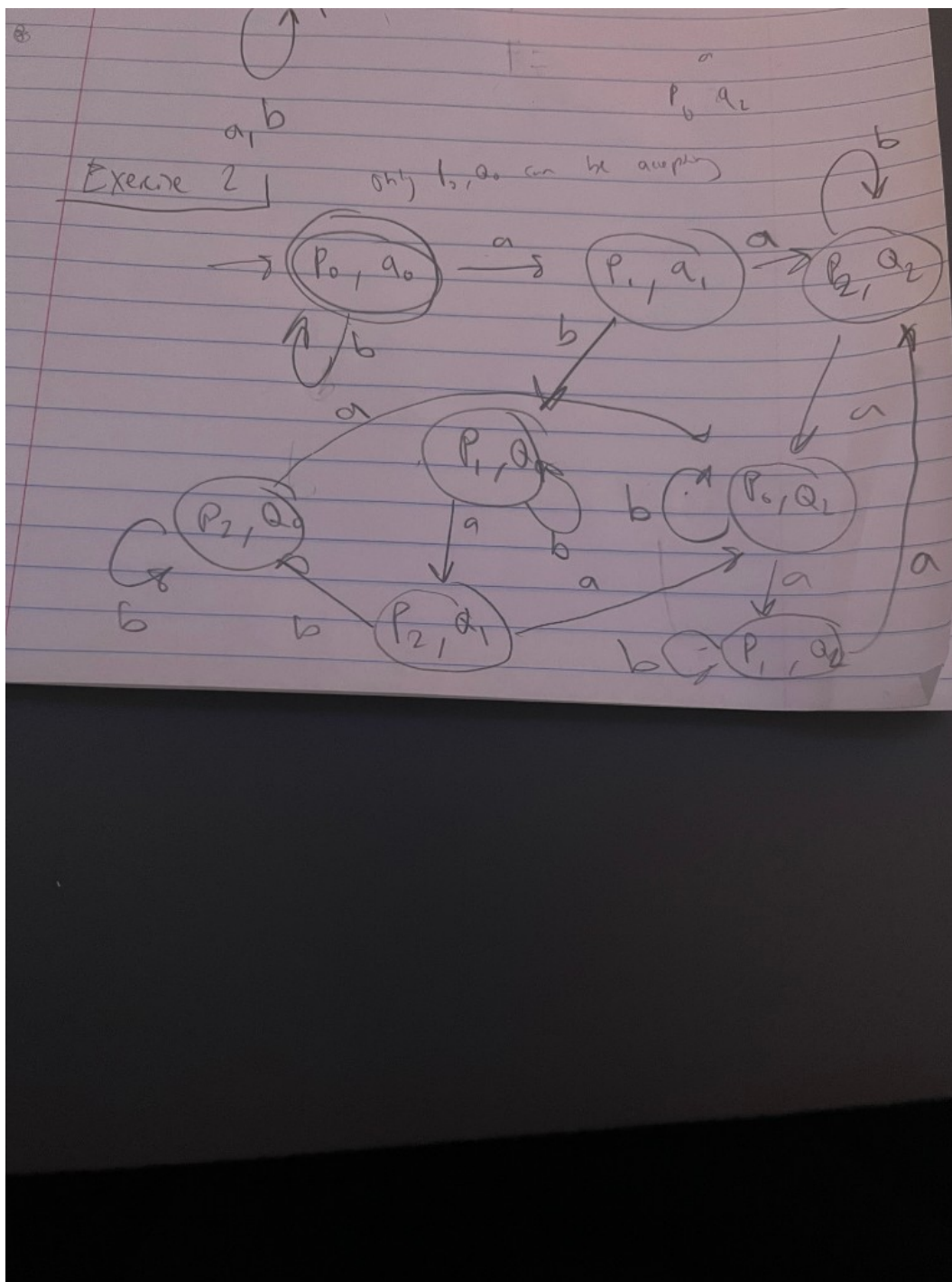


Figure 9: Intersection automaton $\mathcal{B}$ for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ (omit unreachable states).

Note: (P_0, Q_0) as accepting state.

3. Argue that $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$.

Answer: Both automata have the rule that any amount of b 's is allowed. But we must combine the rules regarding a 's. $L(\mathcal{B}^{(1)})$ requires 3 a 's to be included at a time per word, while $\mathcal{B}^{(2)}$ requires the rule that a must be followed by a b . This gives us the conclusion that words that do include an a must come in pairs of 3 a 's with b following the last a .

4. **How can you change $\mathcal{B}$ to get an automaton $\mathcal{B}'$ with $L(\mathcal{B}') = L(\mathcal{B}^{(1)}) \cup \overline{L(\mathcal{B}^{(2)})}$?**

Answer: The Union is equivalent to its intersection, no possible change.

Exercise 3

Let $\mathcal{A}$ be a DFA and q a state of $\mathcal{A}$ such that from q , every input symbol a leaves us in q (i.e. $\delta(q, a) = q$ for all a). Show by induction on the length of the input that for any input string w , we still end up in q (i.e. $\delta^*(q, w) = q$).

Answer: We show it by induction on the length of w . (Here $\delta^*(q, w)$ is just the state we end up in after starting at q and reading the whole string w .)

When the input has length 0, the string is empty. So we don't read anything and we stay at q . So $\delta^*(q, w) = q$ in that case.

Now suppose we've already shown that for every string of length n , we end up back at q . Take a string w of length $n + 1$. We can write it as $w = va$ (some string v of length n plus one symbol a). After reading v we're in $\delta^*(q, v)$, and by our assumption that's q . Then we read a , and we're in $\delta(q, a)$, which is q by the given rule. So after reading w we're still in q , i.e. $\delta^*(q, w) = q$.

So by induction, for all input strings w we have $\delta^*(q, w) = q$.

Question

If you build the product automaton for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ but change the accepting set to get the *union* $L(\mathcal{B}^{(1)}) \cup L(\mathcal{B}^{(2)})$ instead of the intersection, how many states must be accepting?

2.3 Week 3

Homework 1 (NFA)

1. Describe the language accepted by $\mathcal{A}$ using a regular expression.
2. Specify the NFA $\mathcal{A}$ in the form $(Q, \Sigma, \delta, q_0, F)$.
3. Find all paths in $\mathcal{A}$ for the word $w = abab$; represent in a diagram.
4. Construct the determinization $\mathcal{A}^D$ of $\mathcal{A}$ using the power set construction. Create both a transition table and a graphical depiction of $\mathcal{A}^D$.

Answer (NFA specification). Here is how I write the NFA $\mathcal{A}$ formally:

$$Q = \{0, 1, 2, 3\}, \quad \Sigma = \{a, b, c\}, \quad q_0 = 0, \quad F = \{2, 3\}.$$

For the transition function $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ I use

$$\begin{aligned} \delta(0, a) &= \{0, 1\}, & \delta(0, b) &= \{0\}, & \delta(0, c) &= \emptyset, \\ \delta(1, b) &= \{2\}, & \delta(1, c) &= \{2\}, \end{aligned}$$

and any transition not listed goes to the empty set (i.e. there is no move).

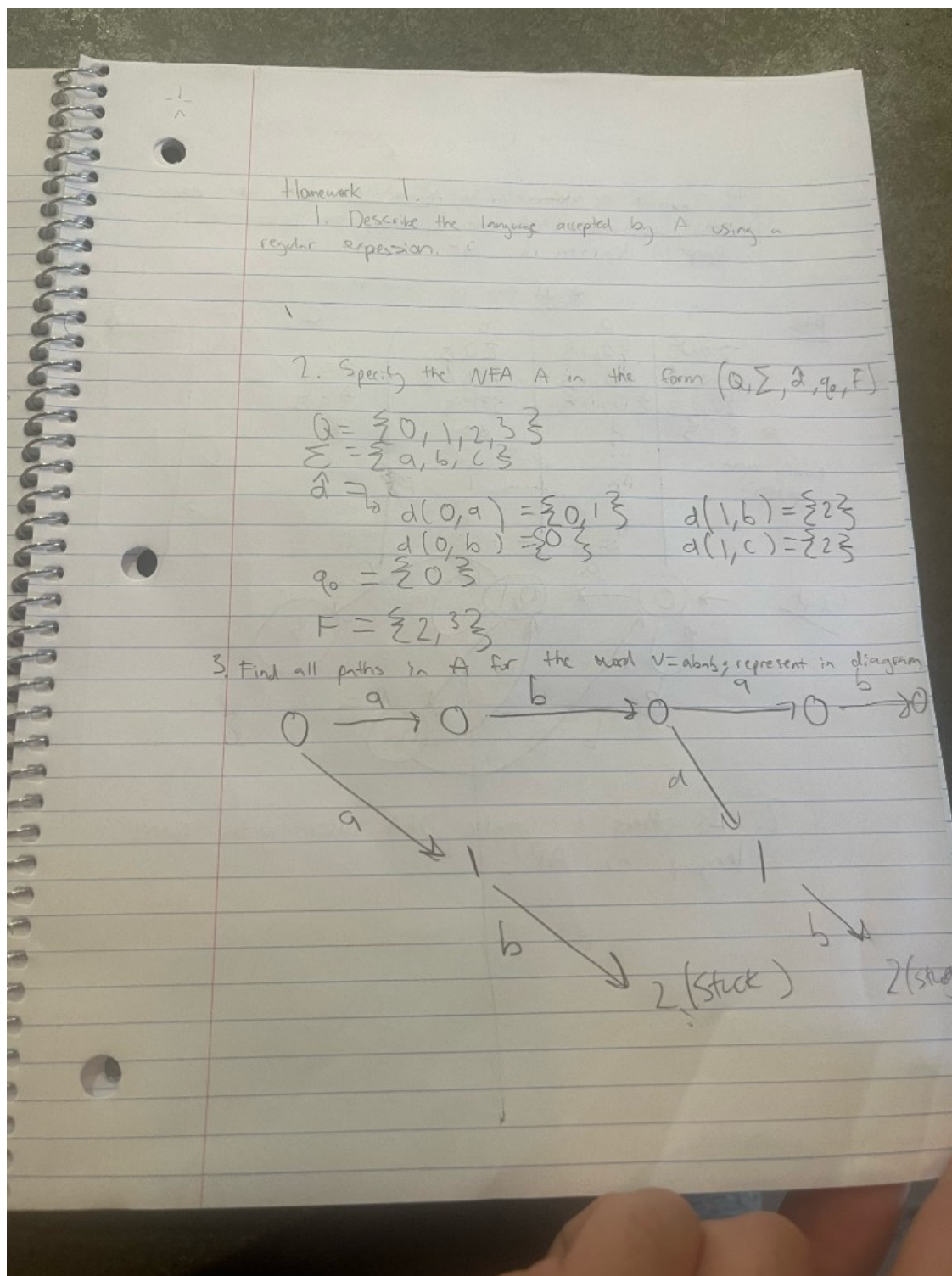


Figure 10: Homework 1: NFA specification $(Q = \{0, 1, 2, 3\}, \Sigma = \{a, b, c\}, q_0 = 0, F = \{2, 3\})$ and all paths for $w = abab$.

Transition table for $\mathcal{A}^D$ over $\{a, b, c\}$. From my construction I focus on the following subset-states:

$$\{0\}, \{0, 1\}, \{0, 2\}, \{3\}.$$

The transitions on a , b , and c are:

State S	on input a	on input b	on input c
$\{0\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$\{0, 1\}$	$\{0, 1\}$	$\{0, 2\}$	$\{3\}$
$\{0, 2\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$

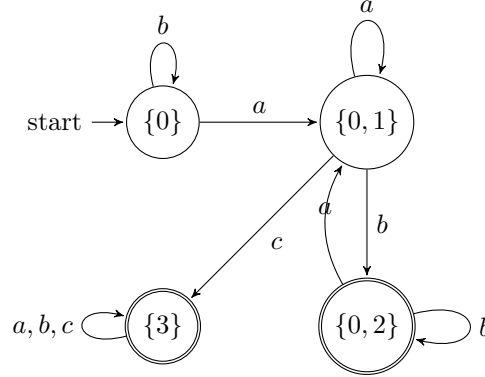


Figure 11: Determinized DFA $\mathcal{A}^D$ for Homework 2 over alphabet $\{a, b, c\}$. Accepting states are the subsets that contain an original accepting state ($\{0, 2\}$ and $\{3\}$).

Homework 2 (DFA and minimization)

Determine $\mathcal{A}^D$ of $\mathcal{A}$ using the power set construction, now with alphabet $\{0, 1\}$. Create both a transition table and a graphical depiction of $\mathcal{A}^D$. Then decide whether there can be a smaller DFA accepting the same language.

Transition table over $\{0, 1\}$. From the NFA given in the homework, I use the following subset-states:

$$q_0, \{q_0, q_1\}, \{q_0, q_1, q_2\}, \{q_0, q_3\}, \{q_0, q_1, q_3\}, \{q_0, q_1, q_2, q_3\}.$$

The determinized DFA $\mathcal{A}^D$ then has this transition table:

State S	on input 0	on input 1
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Any subset that contains the original accepting state(s) is accepting in $\mathcal{A}^D$.

Smaller DFA? Yes. In this table the last two rows (for $\{q_0, q_1, q_2, q_3\}$ and for $\{q_0, q_3\}$) have exactly the same outgoing transitions and the same accepting/non-accepting status. That means these two states are equivalent and can be merged into a single state, giving a smaller DFA that still recognizes the same language.

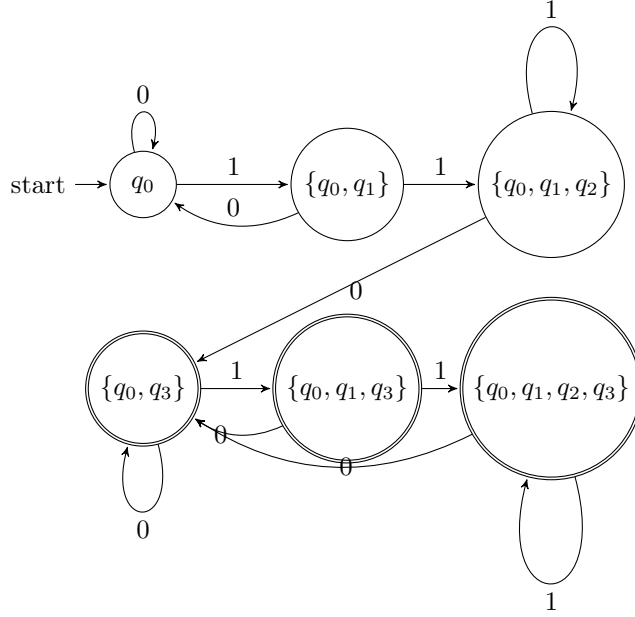


Figure 12: Determinized DFA $\mathcal{A}^D$ for Homework 2 over alphabet $\{0,1\}$. Accepting states are the subsets that contain the original accepting state q_3 .

Question

After building $\mathcal{A}^D$ from an NFA $\mathcal{A}$ using the power set construction, when can we get a smaller DFA that still accepts the same language?

2.4 Week 4

Consider the following DFA $\mathcal{A}$: there is a state p (initial and final) with transition a to itself and transition b to state q ; state q has transition b to itself and transition a to p .

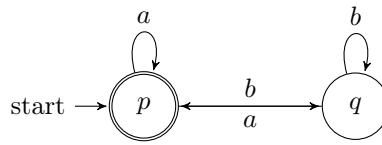


Figure 13: DFA $\mathcal{A}$ for Week 4: p is initial and final; $p \xrightarrow{a} p$, $p \xrightarrow{b} q$, $q \xrightarrow{b} q$, $q \xrightarrow{a} p$.

Question. For $p = 1$ and $q = 2$, compute via Kleene's algorithm all regular expressions $R_{ij}^{(k)}$ and from this a resulting regular expression R describing $L(\mathcal{A})$, i.e., such that $L(R) = L(\mathcal{A})$.

Answer. I label the states so that state 1 is p and state 2 is q . In Kleene's algorithm, $R_{ij}^{(k)}$ denotes the set of all strings that label paths from i to j with intermediate states (if any) in $\{1, \dots, k\}$. The recurrence is

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \cup R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}.$$

I write union as $+$ in expressions below.

Base case $k = 0$ (no intermediate states; only direct edges and ε on same state):

$$R_{11}^{(0)} = \varepsilon + a, \quad R_{12}^{(0)} = b, \quad R_{21}^{(0)} = a, \quad R_{22}^{(0)} = \varepsilon + b.$$

Inductive step $k = 1$ (intermediate state 1 allowed). Using $(R_{11}^{(0)})^* = (\varepsilon + a)^* = a^*$:

$$\begin{aligned} R_{11}^{(1)} &= R_{11}^{(0)} + R_{11}^{(0)} (R_{11}^{(0)})^* R_{11}^{(0)} = (\varepsilon + a) + (\varepsilon + a) a^* (\varepsilon + a) = a^*; \\ R_{12}^{(1)} &= R_{12}^{(0)} + R_{11}^{(0)} (R_{11}^{(0)})^* R_{12}^{(0)} = b + (\varepsilon + a) a^* b = a^* b; \\ R_{21}^{(1)} &= R_{21}^{(0)} + R_{21}^{(0)} (R_{11}^{(0)})^* R_{11}^{(0)} = a + a a^* (\varepsilon + a) = a a^*; \\ R_{22}^{(1)} &= R_{22}^{(0)} + R_{21}^{(0)} (R_{11}^{(0)})^* R_{12}^{(0)} = (\varepsilon + b) + a a^* b = (\varepsilon + b) + a^+ b. \end{aligned}$$

Inductive step $k = 2$ (all states allowed). Set $C = R_{22}^{(1)} = (\varepsilon + b) + a^+ b = b + a^+ b$ (since $\varepsilon + b$ gives ε and b). Then $(R_{22}^{(1)})^* = (b + a^+ b)^*$. Only state 1 is accepting, so $L(\mathcal{A})$ is the set of strings that label paths from 1 to 1:

$$R_{11}^{(2)} = R_{11}^{(1)} + R_{12}^{(1)} (R_{22}^{(1)})^* R_{21}^{(1)} = a^* + a^* b (b + a^+ b)^* a^+.$$

Thus a regular expression R with $L(R) = L(\mathcal{A})$ is

$$\boxed{R = a^* + a^* b (b + a^+ b)^* a^+}.$$

(Here $a^+ = aa^*$; one can also write $a^* b (b \cup a^+ b)^* a^+$ if union is denoted $\cup$.)

Question

If we changed the DFA so that both p and q were accepting states, how would the regular expression R for $L(\mathcal{A})$ change? Would it simplify to a shorter expression?

2.5 Week 5

Exercise (Pumping lemma and non-regular languages)

Show that the following languages are not regular using the pumping lemma:

$$L_1 = \{a^n b^m \in \{a, b\}^* \mid n, m \in \mathbb{N}, n > m \geq 0\},$$

$$L_2 = \{a^{n^2} \in \{a\}^* \mid n \in \mathbb{N}, n \geq 1\},$$

$$L_3 = \{a^p \in \{a\}^* \mid p \in \mathbb{N}, p \text{ prime}\},$$

$$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}.$$

1. Showing L_1 is not regular. I assume L_1 is regular and take a pumping constant p from the lemma. I pick the string

$$w = a^{p+1} b^p \in L_1,$$

since here $n = p + 1$ and $m = p$ so indeed $n > m$. By the pumping lemma I can write $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and for all $i \geq 0$ the pumped string $xy^i z$ should still be in L_1 .

Because $|xy| \leq p$ and w starts with only a 's, the substring y must consist entirely of a 's. So say $y = a^t$ with $t \geq 1$. If I pump down with $i = 0$, I get the string

$$xy^0 z = xz = a^{p+1-t} b^p,$$

so the number of a 's in this string is $p + 1 - t$, and the number of b 's is still p . Since $t \geq 1$, I now have $p + 1 - t \leq p$, so $n \leq m$ for this pumped string. That means $xy^0 z \notin L_1$, contradicting the pumping lemma. So my assumption that L_1 was regular must be wrong; L_1 is not regular.

2. Showing L_2 is not regular. Again I assume L_2 is regular and let p be the pumping length. I choose

$$w = a^{p^2} \in L_2,$$

since p^2 is a perfect square. The lemma gives me a split $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L_2$ for all $i \geq 0$. Because the whole word is just a 's and $|xy| \leq p$, I know that $y = a^t$ for some $1 \leq t \leq p$.

Now I pump once with $i = 2$. Then

$$xy^2z = a^{p^2+t}.$$

If this were in L_2 , I would need $p^2 + t = n^2$ for some integer n . But $1 \leq t \leq p$, so $p^2 < p^2 + t < (p+1)^2 = p^2 + 2p + 1$. There is no perfect square strictly between p^2 and $(p+1)^2$, so $p^2 + t$ cannot be a square. Hence $xy^2z \notin L_2$, contradicting the pumping lemma. So L_2 is not regular.

3. Showing L_3 is not regular. I assume L_3 is regular and take its pumping length p . Choose $w = a^q \in L_3$ where q is any prime number with $q \geq p$ (there are infinitely many primes, so I can definitely pick one large enough). Then $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L_3$ for all $i \geq 0$. As before, $y = a^t$ where $1 \leq t \leq p$.

The length of w is q , and the length of xy^iz is $q + (i-1)t$. I look at $i = q+1$ to pump a bunch of copies of y :

$$|xy^{q+1}z| = q + qt = q(1+t).$$

This number is clearly a composite (product of q and $1+t$, both > 1), so it is not prime. That means $xy^{q+1}z \notin L_3$, contradicting the pumping lemma. Therefore L_3 is not regular.

4. Showing L_4 is not regular. Assume L_4 is regular and let p be its pumping length. I pick

$$w = a^p b^p a^{2p} \in L_4,$$

since here $k = p$ and $m = p$, so the last block is $a^{k+m} = a^{2p}$. The pumping lemma says $w = xyz$ with $|xy| \leq p$, $|y| \geq 1$, and $xy^iz \in L_4$ for all $i \geq 0$. Because $|xy| \leq p$ and the first p symbols are a 's, the piece y sits entirely in the first block of a 's. So $y = a^t$ for some $1 \leq t \leq p$.

If I pump with $i = 2$, the new word is

$$xy^2z = a^{p+t} b^p a^{2p}.$$

In this string, the leftmost block of a 's has length $k' = p + t$, the b -block has length $m' = p$, but the rightmost block is still a^{2p} , i.e. length $2p$. For this to be in L_4 I would need the last block to have length $k' + m' = (p+t) + p = 2p + t$. That would mean I need $2p = 2p + t$, which is impossible since $t \geq 1$. So $xy^2z \notin L_4$, contradicting the pumping lemma. Hence L_4 is not regular.

Question

In these pumping-lemma proofs I always picked one very specific string w in each language. How much freedom do I actually have in choosing w ? Could I have picked a different family of strings (for example, $a^{p+2}b^{p+1}$ for L_1) and still made the same argument work just as well?

2.6 Week 6

Exercise A (Turing machines)

Write a TM to accept the language of binary strings

$$L = \{10^n \mid n \in \mathbb{N}\}$$

and for the input 10^n it returns the string 10^{n+1} . Write a TM to accept the same language L and for input 10^n it returns the string 1. Write a TM to accept the total language of binary strings and, for every string, it returns the string with 0's and 1's swapped.

Conventions. My tape alphabet is $\Gamma = \{0, 1, \sqcup\}$ and the head starts on the leftmost symbol of the input. When I say the machine “returns” an output string, I mean it halts with the output written on the tape (starting at the leftmost cell).

1) TM for $L = \{10^n\}$ that outputs 10^{n+1} . Idea: verify the input is 1 followed by only 0's; scan to the first blank and write one more 0.

$$M_1 = (Q, \Sigma, \Gamma, \delta, q_s, q_{acc}, q_{rej})$$

where $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, \sqcup\}$, and the states are

$$Q = \{q_s, q_z, q_{acc}, q_{rej}\}.$$

The transition function δ is:

state / read	write, move	next state
q_s on 1	write 1, R	q_z
q_s on 0	write 0, R	q_{rej}
q_s on $\sqcup$	write $\sqcup$, R	q_{rej}
q_z on 0	write 0, R	q_z
q_z on 1	write 1, R	q_{rej}
q_z on $\sqcup$	write 0, S	q_{acc}

So if the input is 10^n , it halts with 10^{n+1} on the tape. If the input is not in L , it rejects.

2) TM for $L = \{10^n\}$ that outputs 1. Idea: verify the same language, then erase everything to the right of the first 1.

$$M_2 = (Q, \Sigma, \Gamma, \delta, q_s, q_{acc}, q_{rej})$$

with $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, \sqcup\}$, and

$$Q = \{q_s, q_z, q_{back}, q_{clr}, q_{acc}, q_{rej}\}.$$

Transitions:

state / read	write, move	next state
q_s on 1	write 1, R	q_z
q_s on 0	write 0, R	q_{rej}
q_s on $\sqcup$	write $\sqcup$, R	q_{rej}
q_z on 0	write 0, R	q_z
q_z on 1	write 1, R	q_{rej}
q_z on $\sqcup$	write $\sqcup$, L	q_{back}
q_{back} on 0	write 0, L	q_{back}
q_{back} on 1	write 1, R	q_{clr}
q_{clr} on 0	write $\sqcup$, R	q_{clr}
q_{clr} on $\sqcup$	write $\sqcup$, S	q_{acc}

When it accepts, the only nonblank symbol left from the original input is the first 1, so the tape contains output 1.

3) TM on all binary strings that swaps 0 and 1. Idea: sweep left-to-right and flip each bit; accept when we hit blank.

$$M_3 = (Q, \Sigma, \Gamma, \delta, q_s, q_{acc})$$

where $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, \sqcup\}$,

$$Q = \{q_s, q_{acc}\},$$

and δ is:

state / read	write, move	next state
q_s on 0	write 1, R	q_s
q_s on 1	write 0, R	q_s
q_s on $\sqcup$	write $\sqcup$, S	q_{acc}

This accepts every binary string and halts with all bits flipped.

Exercise 1 (Decidable vs. r.e. closure properties)

Which of the following statements is true? If yes, give an argument; if no, give a counterexample.

1. **If L_1 and L_2 are decidable, then so is $L_1 \cup L_2$.**

True. If D_1 decides L_1 and D_2 decides L_2 , then a decider for $L_1 \cup L_2$ is: on input w , run $D_1(w)$ and $D_2(w)$. Accept iff at least one of them accepts. Since both halt on every input, this procedure halts on every input too.

2. **If L is decidable, then so is the complement $\bar{L} := \Sigma^* \setminus L$.**

True. If D decides L , then a decider for $\bar{L}$ is: on input w , run $D(w)$ and flip the answer (accept $\leftrightarrow$ reject). It still halts on every input.

3. **If L is decidable, then so is L^* .**

True. Let D be a decider for L . Given a word w of length n , there are only finitely many ways to split w into a concatenation $w = x_1x_2 \cdots x_t$ (with $t \geq 0$ and each $x_i \in \Sigma^*$; allowing $t = 0$ gives ε). I can decide whether $w \in L^*$ by checking all splits and accepting iff there exists a split where $D(x_i)$ accepts for every piece x_i . This always halts because there are finitely many splits and D always halts.

4. **If L_1 and L_2 are r.e., then $L_1 \cup L_2$ is r.e.**

True. Let R_1 and R_2 be recognizers for L_1 and L_2 . A recognizer for $L_1 \cup L_2$ can dovetail them: on input w , simulate $R_1(w)$ and $R_2(w)$ in parallel (one step of each, alternating). If either one accepts, accept. If $w \in L_1 \cup L_2$, at least one recognizer will accept in finite time, so this machine accepts in finite time.

5. **If L is r.e., then so is $\bar{L}$.**

False. A standard counterexample is the acceptance problem

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}.$$

We know A_{TM} is r.e. (simulate M on w and accept if it accepts), but $\overline{A_{TM}}$ is *not* r.e. (if both were r.e., then A_{TM} would be decidable by running the two recognizers in parallel and waiting for one to accept, which is impossible).

6. **If L is r.e., then so is L^* .**

True. Let R be a recognizer for L . To recognize L^* on input w , I can enumerate all splits of w into pieces $w = x_1 \cdots x_t$ and dovetail all the computations that check “ $x_1 \in L$ and $\cdots$ and $x_t \in L$ ”

(each check is a finite list of runs of R). If for some split every piece is accepted by R , then accept w . If $w \in L^*$, there exists such a split, and all those runs accept in finite time, so this procedure will eventually accept.

Question

When I argue something is decidable vs. only r.e., what is the key difference in the machine behavior I'm relying on? Is it basically always "will it halt on the reject cases"?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, [Title](#), Publisher, Year.